

## 第 4 册 前端交互权限与误操作保护

本册完整收录对应专题的原始内容，保留代码事实、现有限制与改进建议的区分。供开发维护和 AI 项目理解使用，不应直接发布为面向客户的售后政策。

本册专题：认证权限与安全边界；可靠性与误操作处理；前端页面与交互实现

### 认证权限与安全边界

#### 双认证的实际含义

系统有客户和员工两套主体表、登录接口、Cookie 名称和 audience 校验，但共用同一个后端服务与数据库。这不是两台独立认证服务器，也不是企业 SSO。客户 Cookie 为 supportdesk\_customer\_v2，员工 Cookie 为 supportdesk\_staff\_v2，同一浏览器可同时保持两种身份。

注册用户名长度 3 到 64，只接受英文字母、数字、下划线、点和短横线，后端去首尾空格并转小写。显示名去首尾空格且不能为空。企业编号长度 3 到 50，接受小写字母、数字和短横线。注册密码只限制 8 到 20 个字符，没有数字、大写或特殊符号混合要求，也不会裁剪密码空格。前端按 Unicode 字符计数，以减少 emoji 在两端长度不同的问题。

登录保留旧密码兼容，长度允许 1 到 128，不能拿登录 Schema 反推新注册密码规则。密码不明文存储，使用版本化 scrypt：随机盐、n=32768、r=8、p=3，最大内存 64 MiB。比较使用常量时间摘要比较；计算放在线程池，减少阻塞异步主循环。不存在的账号也做近似同等计算，降低响应耗时枚举风险，但这不是完整的侧信道安全证明。

#### 登录令牌与退出

登录成功生成 32 字节随机值的十六进制令牌，浏览器 Cookie 持有原值，数据库只保存 SHA-256 摘要。会话有效期八小时，没有当前代码实现的无感续期。登录时清理过期会话，轮换当前浏览器同类旧令牌。

Cookie 设置 HttpOnly、SameSite=Strict，Secure 由配置控制，本地默认关闭；上线 HTTPS 必须开启。前端 Axios 开启 withCredentials，不把密码或令牌放进 Pinia/localStorage。HttpOnly 不能阻止所有 XSS 后果，但减少 JavaScript 直接读取令牌。

退出只撤销当前请求对应的 audience 令牌，不删除业务消息，也不自动注销这个账号在所有设备的登录。客户退出不撤销员工 Cookie，反之亦然。成功退出会广播不含身份凭证的 storage 事件，其他同类页面跳到登录页；当前广播主要发生在退出流程，不能声称所有跨标签页登录切换都实时同步。

八小时到期、账号停用或企业停用都会影响后续鉴权。前端路由守卫重新查询服务端身份；任何查询失败也可能把用户导向登录页，因此登录页跳转不一定意味着密码错，也可能是后端不可达。

#### 来源检查和频率限制

认证与身份依赖检查 Origin。允许配置的前端来源与后端自身来源；带不允许 Origin 的请求拒绝。没有 Origin 的非浏览器请求不因缺少头被直接拒绝，仍依赖有效身份和业务权限。CORS 控制浏览器跨域，不是服务端鉴权替代品。

注册、登录按来源地址摘要使用数据库共享的十分钟窗口，最多三十次，失败也计数。数据库行锁防止多进程绕过。超限返回 429 和 Retry-After 600。不直接相信客户端伪造的代理头；生产反向代理需要单独设计可信代理，否则多人共用代理可能共享限额。

当前没有验证码、多因素认证、短信验证、密码找回、账号级锁定、风险设备管理、细分员工角色和聊天全局配额。认证限流不是对全部 API 的反滥用系统。

## 多企业数据隔离

客户聊天请求中的 `company_id` 只是选择服务对象，服务端检查其有效性；`conversation_id` 必须同时属于当前客户和该企业。越权与不存在通常统一反馈 404，减少枚举信息。

员工的企业范围来自 `Principal.company_id`，不让列表参数任意扩展到其他企业。订单工具、工单工具从可信会话计算客户和企业；客户跨会话查询工单可以看到自己在同一企业的记录，不限定只能看到当前会话创建的一张工单。

客户网页“我的工单”允许汇总本人不同企业的工单，这是合法产品视图，不等于 Agent 工具跨企业检索。客户历史消息接口按当前客户所有权验证所选会话；聊天写入还额外验证企业一致。

数据库采用客户和企业成对约束、工单到会话及订单联合外键，加上应用查询过滤。没有实现数据库 Row Level Security；使用高权限数据库账号直接 SQL 查询可以绕过应用权限，所以数据库访问本身必须另行保护。

## 模型与工具执行安全

模型只处理给定上下文，工具名与可调用集合按分支限制。工单查询模型只有只读工具，普通建单要求精确匹配已验证参数，修改工具要求草稿、版本和确认一致，订单写入要求本轮唯一真实查询目标。不能用“模型应该遵循提示词”代替这些程序校验。

来源于客户、商品名、工单描述、知识文档和旧工具结果的文字都是业务数据。提示词要求忽略其中越权指令，但模型仍存在误判风险。独立只读知识分支即便生成错误文字，也没有直接写工单的工具；集成售后流程则另外设置明确确认门槛。

客户无权通过修改工具改状态、系统优先级、负责人、关联订单或企业。影响说明写“非常紧急”仍是客户自述，不自动提升后台优先级。人工实际退款与支付不在工具能力中，不能靠一句自然语言启用。

序号和 UUID 都不是授权证据。服务重复查数据库归属，错误 UUID、过期候选、另一企业编号不能绕过工具范围。工具异常也不能被转成伪造成功消息。

## 隐私和外部数据流向

聊天内容及裁剪后的历史可能发送给配置的聊天模型；知识发布时分块发送给向量供应商，检索时查询文本也发送过去；RAG 回答把命中片段提供给聊天模型。启用 LangSmith 后，自动追踪还可能记录输入、输出与工具参数，因此要按供应商数据流向审查，而不是只看本地日志是否脱敏。

数据库保存业务正文、检查点、回执和知识内容。检查点可能包含完整工具结果；业务消息表只存摘要，不代表系统没有在其他持久化位置保存明细。备份、数据库权限和数据保留期都需要覆盖这些位置。

422 访问日志只记录字段位置、错误类型和原因，不主动记录完整请求体；标准 FastAPI 422 响应仍可能包含校验输入，不能把“日志不写正文”当作“任何响应绝不回显输入”。异常堆栈可能包含外部异常信息，本地日志应限制访问，分享前脱敏。

前端目前以 Vue 文本插值显示对话、工单和知识片段，不执行内容中的 HTML/Markdown 命令。这既减少脚本注入面，也解释了部分 Markdown 符号会原样出现。新增富文本渲染必须重新审查 HTML、链接和附件安全，不能默认信任模型生成内容。

## 目前没有覆盖的生产安全能力

本地 Compose 提供开发数据库口令并映射 5432 端口，不能原样当作公网生产配置。没有应用层字段加密、完整秘密管理平台、日志自动轮转脱敏、备份恢复演练、数据删除审批或合规认证。

默认启动服务只绑定 127.0.0.1，但容器端口发布范围需要独立检查。把开发网页和数据库暴露到公网之前，应配置 HTTPS、可信反向代理、最小权限数据库账号、Secure Cookie、真实 CORS 来源、限流和备份。此处是上线前建议，不是已实现清单。

源代码定位：`core/access.py`、`passwords.py`、`services/auth_service.py`、`db/auth_repository.py`、`identity_models.py`、`schema/access.py`、`agent/tools.py`、`frontend/src/lib/auth-events.ts`。

# 可靠性与误操作处理

## 为什么不能只依赖发送按钮变灰

按钮禁用只能防同一个页面的重复点击，不能防浏览器重载、两个标签页、请求超时重发和服务进程中断。系统分别使用请求幂等、会话锁、业务事务、版本检查和安全错误回执解决不同问题，不能拿其中一个替代其他机制。

幂等的意思是相同已识别请求不重复执行副作用，不是所有相似中文问题永远只建一张工单。不同请求键、不同会话或改了内容，可以被系统视为新的请求；用户仍需核对已经产生的结果。

## 聊天请求的幂等记录

浏览器为一次发送生成 `client_request_id`。服务端将可信客户 ID、企业 ID 和该键组合成稳定 UUID5，以此占有 `chat_operations` 记录；另一个客户即使知道原键也不能读取原回执。

记录另有指纹，由消息内容与原会话 ID 的 JSON 计算 SHA-256。相同键配不同内容返回 409，不能用旧键提交新问题。首次请求进入业务流程；再次提交若已有 `response`，直接返回原 `ChatResponse`；若已有安全 `error`，则返回原错误内容并关联当前请求 ID。

如果记录仍为 `running`，可能是另一请求在执行，也可能是进程已中断。代码不会按“等太久了”就推断可以重新执行。它先查工单、修改活动或旧补充回执，有结果就提示已发生；没法确认就保守标记 `unknown`。

写工具在进入关键写入前保存 `write_started`。工单与 `ticket_id` 回执在同一业务事务提交，客户修改与 `update_activity_id` 同事务提交。这样最终回复失败时仍能证明业务已经成功。请求响应保存晚于业务提

交，不能把 HTTP 没有成功返回当成数据库回滚。

## 浏览器的重复请求保护范围

useAgentChat 的 pendingInput 保留当前组件中的原始企业、会话、内容、上下文版本和键。同一上下文再次发送相同文本可以复用原键。若后端明确表示 not\_executed 且 retryable，前端清除旧尝试，允许客户手动开始新尝试。

刷新、关闭页面、组件重建、切换会话或改变文本后，不保证保留旧 pendingInput。尤其“草稿还在”不等于“请求幂等键还在”，因为草稿在 Pinia，而待重试对象在 composable 局部变量。应先查我的工单和历史，再决定重发。

请求失败时只在输入框当前为空的情况下恢复原文本，防止覆盖用户等待期间新输入。若后端证明已经建单或修改，不把写操作文本自动塞回输入框，而是提示查看工单。网络错误无法解析可靠回执时，默认提示核对，而非自动重试。

## 并发与版本冲突

同一已有会话通过 PostgreSQL pg\_try\_advisory\_xact\_lock 串行处理，锁键来自会话 UUID 的哈希。生产路径用独立连接持有锁，业务 Session 中多次 commit 不会提前释放。另一请求拿不到锁立即返回忙碌，不排队悄悄重放。

人工接管、人工回复、恢复 AI 与 Agent 使用同一会话锁，减少“人工正在接管而 AI 同时写入”的竞态。新会话尚没有公共 ID，各请求独立创建，因此两个新请求键同时发送可能创建两段会话；按钮保护和同键幂等仍有必要。

客户修改工单先展示 version，真正写入时按固定顺序锁请求行和工单行，重新检查客户、企业、关闭状态和 expected\_version。如果客服期间更新了工单，客户旧预览不能覆盖新数据，必须查看最新值再确认。

已经改成目标值时可以返回无需修改，不制造多余活动。版本号改变不仅可能因为字段被另一客户流程更改，也可能因为员工状态、优先级或备注更新；因此任何版本冲突都需要重新核对，而不是强制覆盖。

知识发布用 draft\_revision 和文档锁避免两个员工交叉覆盖。旧版本请求返回 409，重新载入后再操作。客服人工回复用员工、会话、客户端键生成稳定消息 UUID，同键同内容返回已存在消息，同键不同内容拒绝。

## 错误响应的两层含义

错误 code 说明为什么没有正常完成；outcome 说明业务副作用是否发生。HTTP 502 只是其中一种通信结果，必须结合 outcome 判断。

outcome 为 not\_executed 表示这次请求没有开始工单写入，不代表当前整个会话从来没有旧工单，也不代表上轮中断必然安全恢复。ticket\_created、ticket\_updated、comment\_added 分别表示已经建立工单、已经修改、旧补充已记录；unknown 表示不能可靠确定。

常见分类：agent\_timeout 对应 504；模型限流或连接不可用对应 503；模型配置异常对应 503；数据库不可用对应 503；结构输出不合法通常为 502；会话忙、工具历史不完整、检查点需要恢复对应 409；当前模型输入窗口超限对应 413。请求字段错误由 FastAPI 返回 422，未登录为 401，越权或不存在通常为 404。

错误正文包含可展示的说明、请求 ID、是否可尝试新请求、业务 outcome 和可选工单号。retryable 不表示后台自动重新执行，也不表示用户应忽略工单核对。

## 具体误操作的处理建议

连续按发送：页面阻止 pending 期间再发送，服务端同键重放不二次运行；如果已经换键，不属于同一次请求保护。不要为了消除错误提示不断改一个字重新发。

发送后没有回复：先等待合理时间，再查看历史消息、我的工单和运行记录。输入气泡是乐观 UI；发送失败也可能已经保存客户问题或创建工单。保留请求 ID，不先删除数据库记录。

超时后刷新：刷新能重新加载页面，但会丢失页面内待重试键，不是安全重试操作。先恢复原历史会话，核对业务结果，再发送新的消息。

错选订单：在建单前明确说换订单，程序应重新定位并废弃旧选择的确认。序号对应当前候选列表，不要按另一个页面排序猜测。若已经建单，只能联系企业按现有工单处理；一句取消准备不会撤销已提交记录。

同名商品多单：必须用编号或展示序号消歧。系统不默认第一单，也不把历史唯一结果当作永远唯一。一次只展示最近五条，需要编号缩小范围。

编辑错工单：修改前会列候选，只有一张也确认；改变目标会清除原编辑草稿。看清编号、字段前后差异，再回复确认修改。已关闭拒绝客户修改。

确认前政策变化：售后预判重新读取命中政策，快照不一致撤销确认并重新展示。没有资料时转人工，不沿用旧模型结论强行建单。

条件迟迟问不完：新预判最多追问两轮，仍不明转人工；普通直接登记路径只要求基本问题与诉求。不要伪造订单号、签收日或商品状况来“让流程通过”。

客服回复成功但列表没更新：发送状态和列表同步已经分开，界面提示同步失败时刷新列表，不重复发送已成功的回复。其幂等键同样只在当前组件运行期间保存。

员工关掉接管窗口：服务端接管不会自动释放。回到“我正在接待”继续，完成后显式恢复 AI。当前没有其他客服强制转派入口，误退出导致长期占用需管理流程处理。

知识分块调错：未保存时关闭抽屉可放弃；保存后属于服务器草稿版本，不能简单用浏览器撤销恢复旧服务器版本。重新生成会覆盖草稿但不动线上片段，应先确认；线上出错可停用，再修正发布。

## 哪些入口没有同等防重保障

聊天写操作、客户创建模拟订单和人工会话回复有明确请求键机制；普通员工手动创建工单、追加内部备注、普通知识文档创建上传没有统一的持久化同键重放协议。按钮禁用能降低误点，但响应丢失后重复操作仍可能重复记录。

现有数据库回执不是“所有系统范围恰好一次”的证明。直接绕过 API 调用内部服务、管理员操作数据库、使用新请求键、多个不同入口操作同一业务，都需要额外约束。未知执行不会自动补偿退款或回滚，因为项目没有这类真实支付副作用。

## 建议但尚未实现的改进

可考虑为未发送草稿提供用户可选的短期安全恢复，并按客户、企业、会话隔离；为页面内 `pendingInput` 增加明确恢复设计；为客服接管增加离线释放和转派；给手动写入口补幂等键；给所有长表单增加路由离开确认；给不确定回执建立管理员核对界面。

这些建议需要在隐私、共享设备、已发生副作用和权限边界之间权衡，不能单纯把所有数据无期限写进 `localStorage`。本次仅记录事实与建议，没有实现这些额外功能。

## 前端页面与交互实现

前端同时承载客户入口和企业工作台，但两者共享技术设施不意味着共享权限。本篇解释页面如何调用服务、缓存如何更新、哪些内容只是浏览器临时状态。会话恢复的完整情景见记忆与会话恢复（第 2 册），服务端授权见认证权限与安全边界（第 4 册）。

### 应用入口和状态分工

`frontend/src/main.ts` 创建 Vue 应用，安装 Pinia、Vue Router、TanStack Vue Query，并加载全局样式和跨标签页认证事件监听。`App.vue` 是页面容器，实际业务入口由 Router 决定。Element Plus 提供基础控件，Vite 插件负责组件和相关 API 的自动导入，因此某些组件没有显式 import，并不表示没有依赖。

Pinia 的 `stores/conversation.ts` 管理当前客户、企业、会话 ID、聊天消息和未发送草稿。它适合在组件离开后仍需暂存的界面状态，不是服务端数据库的替代品。当前草稿并未通过持久化插件保存到磁盘。

TanStack Query 管理来自服务端的身份、列表、详情、人工接管状态等数据。查询键包含业务上下文，让不同企业或不同工单的结果分开缓存。修改成功后的缓存失效会触发重新查询；它不是直接修改数据库，也不保证另一个浏览器立刻收到推送。

页面组件负责显示和交互，`composable` 组织一组有生命周期的动作，`service` 封装接口地址与数据转换，HTTP 客户端统一连接后端。不要把后端返回字段、登录逻辑和所有按钮处理都集中到一个页面里，也不要为了减少代码把服务端数据永久复制到 Pinia，从而形成两套互相冲突的来源。

### 页面路由和实际入口

根地址 / 转到客户企业目录 `/customer/companies`。客户先登录，再选择企业，进入 `/customer/companies/:companyId/chat`。该企业的模拟订单在 `/customer/companies/:companyId/orders`，客户自己的工单在 `/customer/tickets`。后者可以包含客户在不同企业提交的工单，不能把客户工单列表错误地当作单企业列表。

企业员工从 `/staff/login` 登录，`/staff` 默认进入工单页面。工作台包括 `/staff/tickets`、`/staff/customers`、`/staff/knowledge` 和 `/staff/agent-runs`。旧 `/tickets`、`/agent-runs` 地址转到企业端对应页面；`/customer/chat` 转到企业目录。未知地址也会被重定向，而不是显示一个完整的独立 404 页面。

Router 的前置守卫根据页面所属身份请求当前登录信息。这个守卫改善导航体验，不能替代服务端鉴权。身份查询异常也可能导致跳转登录，因此看到登录页并不必然说明密码错误，可能是接口连接失败。

当前客户聊天的实际入口是 `CustomerChatView.vue`，不是仅凭名字最像聊天页的 `SupportWorkspace.vue`。仓库保留了一些旧组件和旧视图，排查问题必须先沿 `router/index.ts` 确认页面引用关系。

Router 使用浏览器历史模式。开发服务器支持直接访问嵌套路由；生产静态服务器也需要把前端页面路径回退到 `index.html`，同时保留 API、静态资源和真实不存在资源的正确处理。否则从站内点击正常，刷新聊天页面却可能得到服务器 404。

## 网络封装与错误反馈

`lib/http.ts` 创建 Axios 实例，从 `VITE_API_BASE_URL` 读取后端地址，启用携带 Cookie，默认超时 60 秒。配置缺失会阻止正常初始化，不能指望自动猜测后端端口。浏览器中的地址必须是浏览器能访问到的地址，而不是仅容器内部可解析的服务名。

`services/agent-service.ts` 负责聊天与历史消息接口，`services/agent-feedback.ts` 将不同错误结果转成客户能理解的文案。特别要区分“请求确定未写入”和“结果未知”：网络断开可能发生在服务端成功之后，前端不能简单显示失败并换一个请求 ID 重发。

聊天、历史恢复和人工状态等相关操作显式控制重试行为，避免把有副作用的请求交给通用自动重试。其他列表查询有各自的 Query 配置，不能据此推断整个应用所有请求都永远不重试。若调整全局 Query 默认值，要重新审查各 mutation，而不是只验证列表刷新。

后端响应的请求 ID 是排错线索，不是登录凭证，也不是业务完成回执。向开发者反馈时，应同时给出操作时间、页面、请求 ID 和可见错误；无需粘贴密钥或完整 Cookie。

## 聊天输入和发送生命周期

`useChatKeyboard.ts` 统一处理 Enter 发送和 Shift+Enter 换行，并避开输入法组合输入过程。中文输入时按回车选词不应被当作发送，这是键盘处理必须保留的边缘条件。输入快捷提示只设置草稿，仍由客户确认发送，不应将浏览页面误变成提交业务。

`useAgentChat.ts` 是聊天交互的主要协调层：准备请求、调用接口、显示忙碌状态、处理错误、恢复历史，并跟随人工状态变化更新页面。它不是另一个 AI 模型，也不替代 LangGraph。

发送前会保存本次输入及请求标识，再进入等待状态。重复点击在界面层受到限制，服务端还会独立执行幂等和会话锁检查。界面防抖只是便利措施，不能作为防止重复建单的唯一依据。

等待期间，历史轮询不能覆盖当前临时消息。异步请求返回时还会检查上下文版本，避免客户已切换企业后，把原企业的回答插入新页面。这个检查只防止页面污染，不会撤销已经发到服务端的请求。

当前客户会话的历史消息和人工接管状态按约 3 秒轮询，因此人工回复不是通过 WebSocket 实时推送。网络、后台标签页节流和服务端耗时都会影响看到新消息的时间。没有消息时的欢迎介绍是界面内容，不能直接当作数据库中已经保存的一条 `AIMessage`。

客户聊天以普通文本呈现回答，并显示对应工具反馈或知识来源。当前不能把模型输出的 Markdown 语法都视为有格式富文本，更不能把模型文字当作可信 HTML 插入页面。知识引用是否真正支持回答仍由检索和生成逻辑决定，显示引用不等于事实绝对正确。

## 离开页面和恢复草稿

客户草稿以企业为单位保存在当前运行中的 Pinia 内存。客户从聊天跳去模拟订单或我的工单，再返回同企业，通常可以继续原草稿；切换企业时，原企业草稿和目标企业草稿分别处理。

这不是按每个 conversation\_id 保存的草稿。客户在同一企业切换历史会话时，未发送文本可能继续留在输入框。因此恢复历史后应先核对输入内容，不能假设它一定属于新选中的旧会话。

刷新、关闭标签页、浏览器崩溃会重建内存状态，当前没有可靠的草稿恢复保证。退出客户账号会清空相关界面状态，避免后一位登录者看到上一位客户草稿。跨标签页的退出事件用于清理认证相关状态，不是多标签页草稿同步协议。

useAgentChat 内部用于重试的待发送信息也有生命周期：组件卸载或刷新后，不保证保留原请求标识。出现“结果未知”后离开页面再返回，应先核对工单和历史记录。不要将“输入框里仍有同样的字”误认为“系统仍能识别这是同一笔请求”。

若未来需要刷新恢复，应新增有身份和企业隔离、过期清理、版本兼容及退出清除的本地草稿设计，并考虑敏感内容存储风险。该能力目前是建议，不是已实现。

## 模拟订单与工单页面

CustomerOrdersView.vue 提供模拟订单列表和创建表单。商品名、金额、状态、签收日期由客户填写，前端做便利校验，后端再次校验。只有已完成状态允许签收日期；切换到其他状态时会清空日期，防止残留的隐藏字段影响售后判断。

订单创建使用本地请求指纹与请求 ID 协调重试。成功后清理相关表单、返回列表前页并刷新查询。刷新导致本地尝试记录丢失时，仍应先查订单列表，不能保证再次填写同样内容一定被识别为原请求。

CustomerTicketsView.vue 展示客户工单，企业端的 TicketCenter.vue 配合 useTicketCenter.ts 提供筛选、详情、状态处理和内部备注。共享的 TicketIssueDetails.vue 统一问题信息展示，避免两端对“订单信息”“问题描述”“售后诉求”的理解发生偏差。

客户看到的修改和补充记录由 CustomerTicketComments.vue 读取客户可见接口。企业内部备注不应通过复用企业详情响应泄露给客户。TicketConversation.vue 展示企业有权查看的关联会话，不把消息写进客户正在使用的 Pinia 聊天状态。

部分写操作会在成功后等待相关缓存刷新，因此按钮的等待状态可能包含刷新耗时，而不仅是写接口耗时。应结合网络请求判断是否真的卡住。前端成功提示不能替代后端状态机或版本冲突处理。

## 人工接管入口与回复

StaffCustomersView.vue 包含明显的人工待接管区域，HandoffInbox.vue 展示待接管和我接管的会话；StaffNavigation.vue 轮询提示数量。员工无需先猜客户姓名再逐层进入历史，才能发现求助。

HandoffControls.vue 根据会话状态和当前负责人决定是否允许接管、回复或交回 AI。人工回复保留本次提交标识用于处理重试，发送状态与后续刷新分开收尾，避免回复已经发送但按钮一直显示发送中。它仍依赖服务端鉴权和锁，按钮禁用并不构成安全边界。

员工查看客户历史消息时，页面通过复制后反转数组把最近消息放在上方，不直接倒置共享查询缓存。客户正常聊天窗口仍遵循聊天阅读顺序；不能把一个页面的展示习惯推广为所有接口都倒序返回。

客服离开页面不会自动交回 AI。服务端仍保留人工接管状态；客户之后发送的内容会继续进入该人工会话。当前没有员工离线检测后自动释放、轮班自动转派或无人响应自动升级机制。

## 知识库编辑页面

`KnowledgeView.vue` 组织资料列表、录入、上传、详情、检索测试和历史工单经验入口。

`KnowledgeDraftEditor.vue` 管理本地分块副本，支持调整内容、拆分和合并，并带修订号提交，避免将旧版本静默覆盖新版本。

原文、尚未保存的编辑、副本已经保存的草稿、已经发布的检索索引，是不同阶段。客户只能检索有效发布数据，不能因为管理员在浏览器中修改了一段文字就立刻看到新规则。

关闭详情时存在针对未保存编辑的确认；重新分块会提醒覆盖编辑。发布按钮受未保存状态、空内容和案例审核状态约束。但当前没有覆盖所有路由离开、浏览器关闭、断电情景的通用草稿持久化，因此编辑长资料应及时保存。

上传或创建资料不具备与聊天建单完全相同的幂等协议。响应丢失后先看列表，避免重复录入。员工确认历史经验审核代表进行了业务和隐私检查，不能用勾选框代替实际脱敏。

## 运行记录与可观测性界面

`AgentRunCenter.vue` 配合 `useAgentRunCenter.ts` 展示运行列表、统计和详情。列表与统计约 10 秒更新一次，选中记录才请求详情。当前详情不是随节点执行逐帧更新的实时调试器。

详情中的步骤由后端追踪器在本次运行结束阶段保存，早期记录可能没有这类数据；追踪器只记录已经纳入观察范围的节点与工具，不代表每条内部函数调用都列出。

统计字段 `tool_calls` 当前统计的是“带有工具名称的运行记录数量”，不是全部工具调用事件的总和。同一轮多次调用工具不能据此精确计次，后续若做运营报表应先明确统计口径。

企业侧导航采用适应窄屏的布局：中等宽度可压缩为图标导航，小屏改变排列方式，长名称限制溢出。这些样式改善可用性，不等于已经完成完整的无障碍或所有移动设备验收。

## 前端验收时要主动检查的边缘情况

验收至少应覆盖中文输入法回车、Shift+Enter、空消息、连续点击发送、请求中切换企业、同企业切换历史、刷新后找回服务端会话、退出再登录另一账号、客服接管后离开页面、两个员工争抢同一会话、编辑知识草稿后关闭详情，以及写入成功但刷新失败。

判断成功不能只看按钮颜色。聊天要核对消息及工具回执，工单要核对编号和实际字段，人工回复要在客户侧看到消息，发布知识要用独特事实检索确认。对暂未支持的恢复情景，应清楚告知边界，而不是把数据丢失称为用户操作错误。